

◆ LINQ (Language Integrated Query) – Overview

LINQ allows you to perform query operations directly in C# over data sources like:

- Collections (List, Array, Dictionary)
- Databases (via LINQ to SQL/Entity Framework)
- XML (LINQ to XML)
- DataSets, Files, etc.

LINQ = SQL-style querying + strongly typed syntax + IntelliSense support

◆ Real-World Use Cases

Use Case	Description
✓ Filtering Products	Get all available products under a price or category.
✓ Joining Data	Join Customers with their Orders (like SQL JOIN).
✓ Grouping & Aggregation	Group employee salaries by department and find averages.
✓ Projection	Create DTOs or select specific fields.
✓ Complex Search	Multi-field keyword search (e.g., in CRM or inventory systems).
✓ Paginated Data	Use <code>.Skip().Take()</code> for pagination like Grid/Blazor/UIs.
✓ Detect Duplicates	Group by a field and count > 1.
✓ Generate Reports	LINQ-to-Entities or LINQ-to-Objects for summary reports.

◆ Best Practices for Using LINQ

1. Avoid overusing LINQ in performance-critical sections

- Prefer `foreach` in high-frequency loops if LINQ causes allocations.

2. Use method syntax for complex queries

- Easier to compose dynamically (`Where()`, `Select()`, `GroupBy()` etc.)

3. Use `ToList()/ToArray()` only when needed

- Avoid early materialization unless required. Use deferred execution.

4. Null safety

- Always check for nulls to avoid `NullReferenceException`.

5. Combine with `let`, `into`, `join` smartly in query syntax

6. Leverage LINQ with `async` only when supported (e.g., `ToListAsync()` in EF Core).

7. Avoid multiple enumerations of the same LINQ result.

8. Don't misuse LINQ for side effects (like `Select(x => DoSomething(x))` without consuming the result).

Hands-On Code Examples (with .NET 8)

Filter products by category and price

csharp

```
var products = new List<Product>
{
    new Product { Id = 1, Name = "Laptop", Category = "Electronics",
Price = 60000 },
    new Product { Id = 2, Name = "Phone", Category = "Electronics",
Price = 30000 },
    new Product { Id = 3, Name = "Shirt", Category = "Clothing", Price
= 2000 }
```

```
};

var result = products
    .Where(p => p.Category == "Electronics" && p.Price < 50000)
    .ToList();
```

2 Group by Department and calculate average salary

csharp

```
var employees = new List<Employee>
{
    new Employee { Name = "John", Department = "HR", Salary = 50000 },
    new Employee { Name = "Jane", Department = "IT", Salary = 70000 },
    new Employee { Name = "Bob", Department = "IT", Salary = 65000 },
};

var avgSalaries = employees
    .GroupBy(e => e.Department)
    .Select(g => new
    {
        Department = g.Key,
        AverageSalary = g.Average(e => e.Salary)
    })
    .ToList();
```

3 Join Customers and Orders (like SQL Inner Join)

csharp

```
var customers = new List<Customer>
{
    new Customer { Id = 1, Name = "Alice" },
    new Customer { Id = 2, Name = "Bob" }
};

var orders = new List<Order>
```

```
{
    new Order { Id = 1, CustomerId = 1, Product = "Phone" },
    new Order { Id = 2, CustomerId = 1, Product = "Laptop" }
};

var customerOrders = customers
    .Join(orders,
        c => c.Id,
        o => o.CustomerId,
        (c, o) => new { c.Name, o.Product })
    .ToList();
```

4 Paginated Records (Skip/Take)

csharp

```
int pageSize = 5, pageIndex = 2;

var pagedResult = employees
    .Skip((pageIndex - 1) * pageSize)
    .Take(pageSize)
    .ToList();
```

5 Expression-based Search (Search CRM)

csharp

```
string keyword = "john";

var result = employees
    .Where(e => e.Name.Contains(keyword,
StringComparison.OrdinalIgnoreCase)
        || e.Department.Contains(keyword))
    .ToList();
```

◆ LINQ in Entity Framework (LINQ-to-Entities)

csharp

```
var activeUsers = await _context.Users
    .Where(u => u.IsActive && u.Role == "Admin")
    .OrderBy(u => u.Name)
    .Select(u => new { u.Id, u.Name })
    .ToListAsync();
```

Use `ToListAsync()`, `FirstOrDefaultAsync()` etc. in EF Core with LINQ.

◆ Interview Questions & Answers (With Explanation)

✓ Q1: What is the difference between **IEnumerable** and **IQueryable**?

Answer:

- **IEnumerable**: Works in memory (LINQ-to-Objects). Evaluation is client-side.
- **IQueryable**: Works with external data sources like DBs (LINQ-to-Entities). Query is translated to SQL and executed on the DB server.

csharp

```
List<Product> localProducts = GetProducts();
IEnumerable<Product> result = localProducts.Where(p => p.Price >
1000);

DbSet<Product> dbProducts = _context.Products;
IQueryable<Product> result2 = dbProducts.Where(p => p.Price > 1000);
// Translates to SQL
```

✓ Q2: Explain deferred vs immediate execution in LINQ

Answer:

- **Deferred:** Query is not executed until iterated (`Where`, `Select`, `GroupBy`).
- **Immediate:** Executed instantly (`ToList()`, `ToArray()`, `Count()` etc.)

csharp

```
var query = employees.Where(e => e.Salary > 50000); // deferred  
var result = query.ToList(); // executed now
```

✓ Q3: How to improve LINQ performance?

Answer:

- Use `.Any()` instead of `.Count() > 0`
 - Use `.Select()` before `.ToList()` to reduce memory usage
 - Avoid chaining too many `Select()` or `Where()` if not needed
 - Use `.AsNoTracking()` in EF if you don't need change tracking
 - Avoid complex joins in memory. Do them in DB (LINQ-to-Entities)
-

✓ Q4: Can we write custom LINQ extension methods?

Answer:

Yes! You can define custom extension methods over `IEnumerable<T>` to reuse logic.

csharp

```
public static class EmployeeExtensions  
{  
    public static IEnumerable<Employee> HighEarnings(this  
IEnumerable<Employee> employees, decimal limit)  
    {  
        return employees.Where(e => e.Salary > limit);  
    }  
}
```

```
}
```

```
// Usage:  
var top = employees.HighEarnings(70000);
```

✅ **Q5: What is the difference between `First()`, `FirstOrDefault()`, `Single()` and `SingleOrDefault()`?**

Method	Behavior
<code>First()</code>	Throws exception if no element
<code>FirstOrDefault()</code>	Returns default (null) if not found
<code>Single()</code>	Expects exactly 1 match, else throws
<code>SingleOrDefault()</code>	Returns null if none, throws if more than 1

Summary

Feature	LINQ
Type-Safety	✅ (compile-time check)
IntelliSense	✅
Easy to Read	✅
SQL-style Querying	✅
Composable	✅
Works with DBs and In-Memory	✅

Let's begin with a **Mini Project: Student Management System** using **LINQ + EF Core (.NET 8)**.

Mini Project: Student Management System

Project Summary:

Build a simple system to manage students, courses, and enrollments using:

- **Entity Framework Core 8**
 - **LINQ queries** for filtering, grouping, joining, and projections
-

Entities (Models)

csharp

```
public class Student
{
    public int StudentId { get; set; }
    public string Name { get; set; }
    public ICollection<Enrollment> Enrollments { get; set; }
}

public class Course
{
    public int CourseId { get; set; }
    public string Title { get; set; }
    public ICollection<Enrollment> Enrollments { get; set; }
}

public class Enrollment
{
    public int EnrollmentId { get; set; }

    public int StudentId { get; set; }
```

```
public Student Student { get; set; }

public int CourseId { get; set; }
public Course Course { get; set; }

public decimal Grade { get; set; }
}
```

DbContext

csharp

```
public class SchoolContext : DbContext
{
    public DbSet<Student> Students { get; set; }
    public DbSet<Course> Courses { get; set; }
    public DbSet<Enrollment> Enrollments { get; set; }

    protected override void OnConfiguring(DbContextOptionsBuilder
optionsBuilder)
    {
        optionsBuilder.UseSqlServer("Server=.;Database=SchoolDb;Trusted_Connec
tion=True;TrustServerCertificate=True");
    }
}
```

Step-by-Step Demo Logic

1. Add Sample Data (Seeding)

csharp

```
using var context = new SchoolContext();

context.Database.EnsureDeleted();
```

```
context.Database.EnsureCreated();

var s1 = new Student { Name = "Arun" };
var s2 = new Student { Name = "Priya" };

var c1 = new Course { Title = "C#" };
var c2 = new Course { Title = "SQL Server" };

var enrollments = new List<Enrollment>
{
    new Enrollment { Student = s1, Course = c1, Grade = 85 },
    new Enrollment { Student = s1, Course = c2, Grade = 90 },
    new Enrollment { Student = s2, Course = c2, Grade = 95 }
};

context.AddRange(enrollments);
context.SaveChanges();
```

2. LINQ Queries

a. Get all students with their courses and grades

csharp

```
var data = context.Students
    .Select(s => new
    {
        s.Name,
        Courses = s.Enrollments.Select(e => new
        {
            e.Course.Title,
            e.Grade
        })
    }).ToList();

foreach (var student in data)
{
    Console.WriteLine($"Student: {student.Name}");
}
```

```
        foreach (var course in student.Courses)
            Console.WriteLine($"    {course.Title} - {course.Grade}");
    }
```

b. Average grade per course

csharp

```
var courseAverages = context.Enrollments
    .GroupBy(e => e.Course.Title)
    .Select(g => new
    {
        Course = g.Key,
        AverageGrade = g.Average(e => e.Grade)
    }).ToList();

foreach (var avg in courseAverages)
    Console.WriteLine($" {avg.Course}: Avg Grade =
{avg.AverageGrade}");
```

c. Students scoring above 90 in any course

csharp

```
var topScorers = context.Enrollments
    .Where(e => e.Grade > 90)
    .Select(e => new
    {
        e.Student.Name,
        e.Course.Title,
        e.Grade
    }).ToList();

foreach (var entry in topScorers)
    Console.WriteLine($" {entry.Name} - {entry.Title} :
{entry.Grade}");
```

d. Course-wise Enrollment Count

csharp

```
var courseEnrollments = context.Courses
    .Select(c => new
    {
        c.Title,
        Count = c.Enrollments.Count
    }).ToList();

foreach (var ce in courseEnrollments)
    Console.WriteLine($"{ce.Title}: {ce.Count} enrolled");
```

Project Highlights

Feature	LINQ & EF Usage
Projection	<code>.Select()</code>
Join-like logic	Navigation properties + projection
Aggregation	<code>.GroupBy()</code> , <code>.Average()</code>
Filtering	<code>.Where()</code>
Deferred Execution	Used when required
Real DB	EF Core with SQL Server

Optional Enhancements

- Use `DTOs` or ViewModels
- Add ASP.NET Core Web API / Blazor frontend
- Implement CRUD
- Add pagination with `.Skip()` & `.Take()`